

Software Testing

Zeeshan Ali Rana

Software Faults and Failures

Why Does Software Fail?

- **Fault (bug):**

Programming or design error whereby the delivered system does not conform to specification (e.g., coding error, protocol error)

- **Failure:**

Software does not deliver the service expected by the user (e.g., mistake in requirements, confusing user interface)

Software Faults and Failures

Why Does Software Fail?

- Wrong requirement: not what the customer wants
- Missing requirement
- Requirement impossible to implement
- Faulty design
- Faulty code
- Improperly implemented design

Objective of Testing

- Objective of testing: discover faults
- A test is successful only when a fault is discovered
 - Fault identification is the process of determining what fault caused the failure
 - Fault correction is the process of making changes to the system so that the faults are removed



Elements of a Test Case

- Purpose
- Input
- Expected Output
- Actual Output
- Sample Format:

Test Case ID	Purpose	Input Data	Expected Output	Actual Output	Result

Types of Faults

- Algorithmic fault (logic)
- Computation and precision fault
 - a formula's implementation is wrong
- Capacity or boundary faults
 - System's performance not acceptable when certain limits are reached
- Timing or coordination faults
- Performance faults
 - System does not perform at the speed prescribed

Software Faults and Failures

Typical Algorithmic Faults

- An algorithmic fault occurs when a component's algorithm or logic does not produce proper output
 - Branching too soon
 - Branching too late
 - Forgetting to initialize variable or set loop invariants
 - Comparing variables of inappropriate data types

Different Level of Failure Severity

- Catastrophic: causes death or system loss
- Critical: causes severe injury or major system damage
- Marginal: causes minor injury or minor system damage
- Minor: causes no injury or system damage

Testing Issues

Who Performs the Test?

- Independent test team
 - avoid conflict
 - personal responsibility vs need to discover faults
 - improve objectivity
 - allow testing and coding concurrently

Testing Issues

Views of the Test Objects

- Closed box or black box: functionality of the test objects
 - Equivalence Class, Boundary Value Analysis, Scenario-based, Decision Table based, State Machine based...
- Clear box or white box: structure of the test objects
 - Control Flow
 - Basis Path, Branch, Statement, Decision...
 - Data Flow
 - Du Path, All-uses Path

Test Cases for Triangle Problem

Test cases based on input range

TC Id	What to test	A	B	C	EO
1	EC ₁ of A	-10	15	15	Value of A not in valid range
2	EC ₁ of B	15	-10	15	Value of B not in valid range
3	EC ₁ of C	15	15	-10	Value of C not in valid range
4	EC ₃ of A	-205	15	15	Value of A not in valid range
5	EC ₃ of B	15	-205	15	Value of B not in valid range
6	EC ₃ of C	15	15	-205	Value of C not in valid range
7	EC ₂ of A, B, C	15	15	15	Equilateral

Note that we have made only one test case instead of three separate test cases to check ABC in valid range. This one test could be any test cases with any valid combination of A, B, C; for example see any of the valid range based test cases.

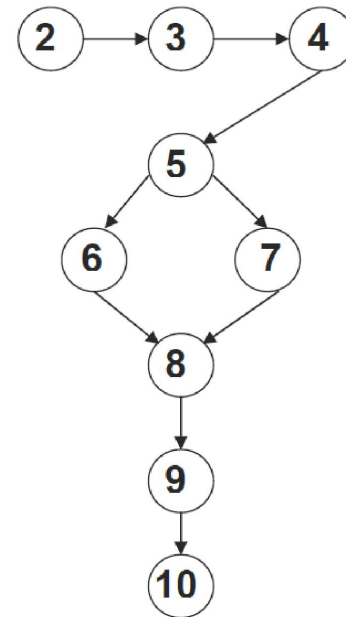
Test cases based on equivalence classes within valid range

TC Id	What to test	A	B	C	EO
1	EC 1	15	15	15	Equilateral
2	EC 2	10	20	10	Isosceles
3	EC 3	5	6	7	Scalene
4	EC 4	1	2	10	Not a triangle

Whitebox Testing

Control Flow Graph

1. Program 'Simple Subtraction'
2. Input (x, y)
3. Output (x)
4. Output (y)
5. If $x > y$ then DO
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output (z)
10. Output "End Program"

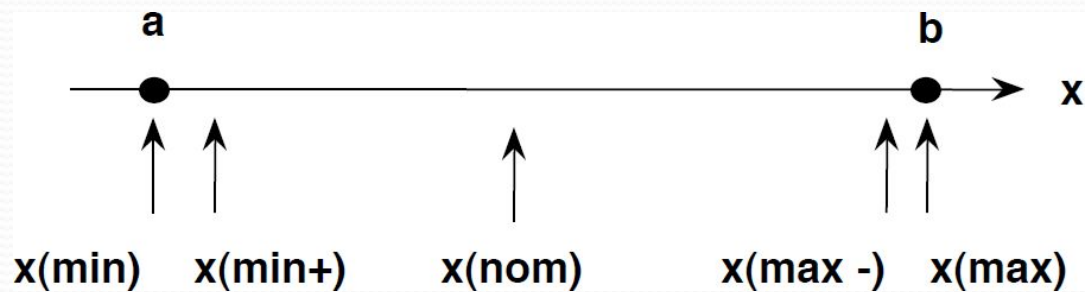
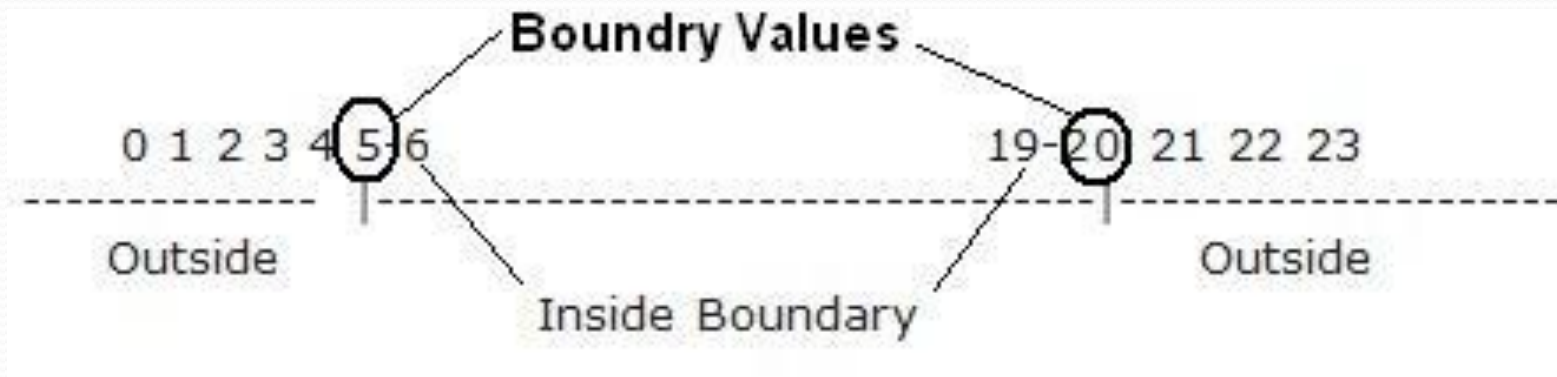


Statement testing
Branch testing
Path testing

...

$$\text{Cyclomatic Complexity} = E - N + 2 = 9 - 9 + 2 = 2$$

Boundary Value Analysis



Example

- An Address text box which allows maximum 500 characters. So, writing test cases for each character once will be very difficult so that will choose boundary value analysis.

Example

- An Address text box which allows maximum 500 characters. So, writing test cases for each character once will be very difficult so that will choose boundary value analysis.
- No character
- 1 character
- 2 characters
- 499 characters
- 500 characters
- 501 characters

Equivalence and Boundary Value

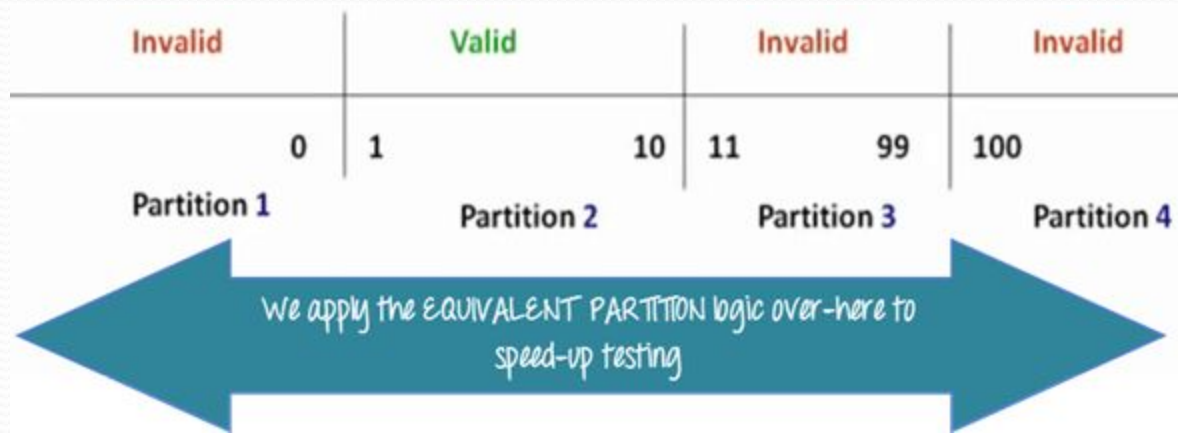
- Let's consider the behavior of Order Pizza Text Box
Pizza values 1 to 10 is considered valid. A success message is shown.

Here is the test condition

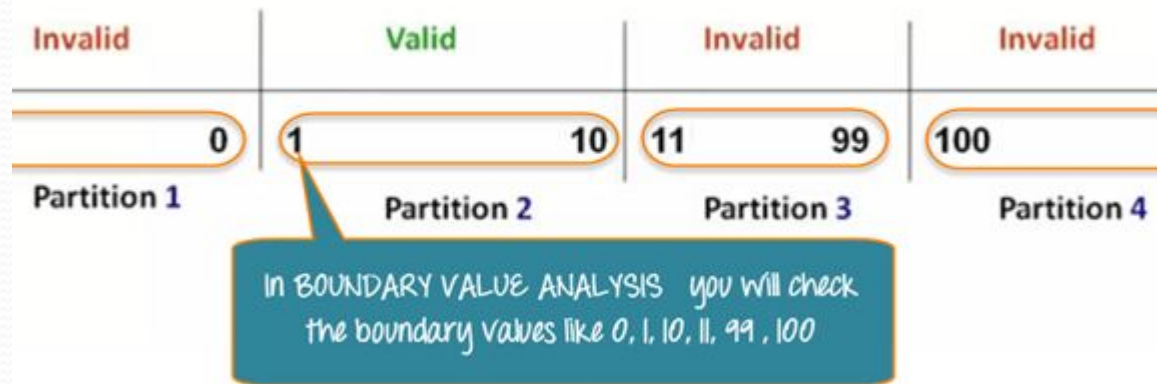
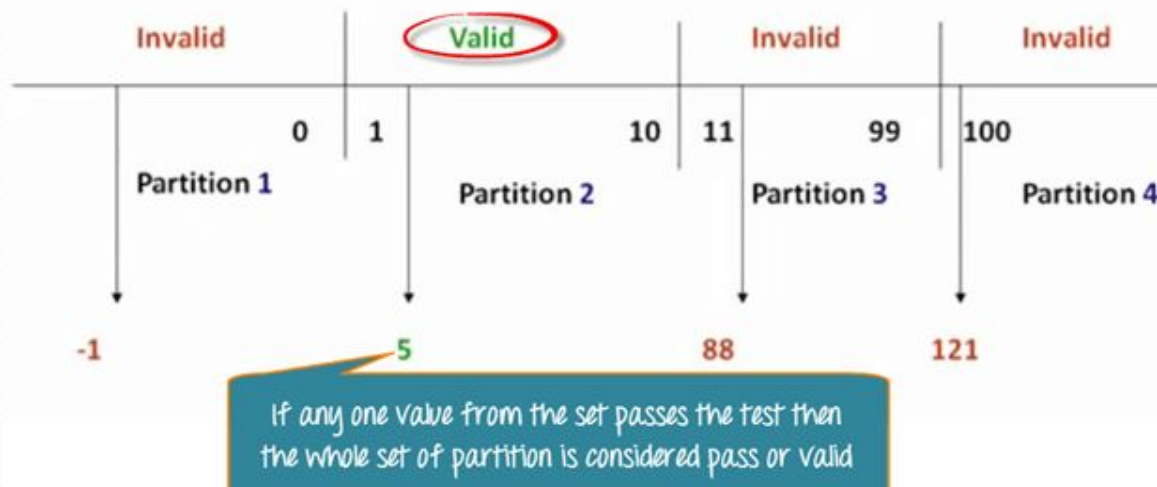
Any Number greater than 10 entered in the Order Pizza field(**let say 11**) is considered **invalid**.

Any Number less than **1** that is **0 or below**,
then it is considered invalid.

Numbers 1 to 10 are considered valid Any **3 Digit Number say -100** is invalid.



Equivalence and Boundary Value

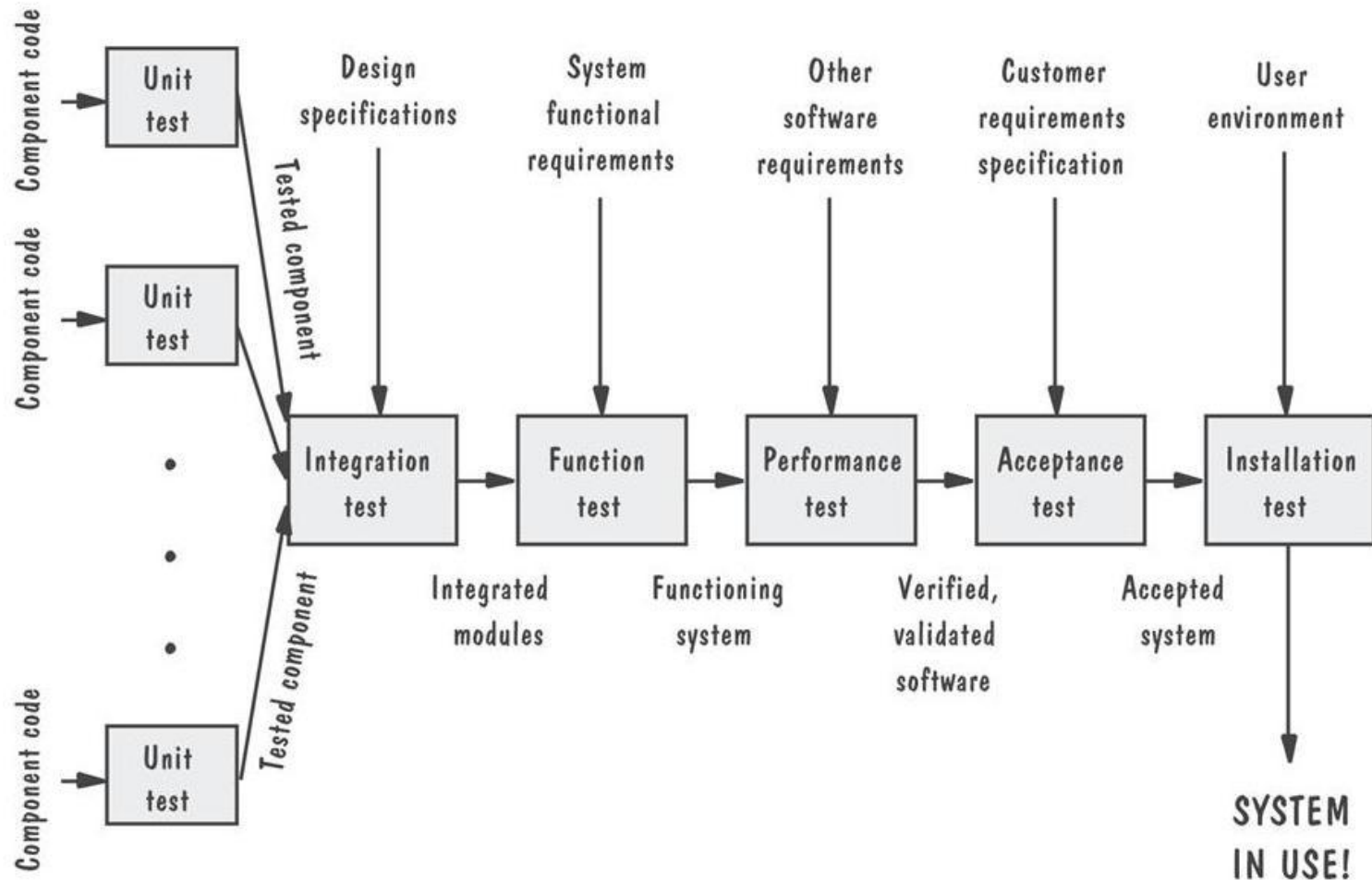


Levels of Testing

- Module testing, component testing, or unit testing
- Integration testing
- System Testing
 - Function testing
 - Performance testing
- Acceptance testing
- Installation testing

Levels of Testing

Testing Organization Illustrated



Unit Testing

Code Review

- Code walkthrough
 - In a **walkthrough**, you present your code and accompanying documentation to the review team, and the team comments on their correctness.
- Code inspection
 - In an **inspection**, the review team checks the code and documentation against a prepared list of concerns. It is more formal.

Unit Testing

Steps in Testing

- Determining test objectives
- Selecting test cases
- Executing test cases

Unit Testing

- **Statement testing:** Every statement in the component is executed at least once in some test.

$$\text{Statement coverage (\%)} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} \times 100$$

- **Branch testing:** For every decision point in the code, each branch is chosen at least once in some test.

$$\text{Branch coverage (\%)} = \frac{\text{Number of executed branches}}{\text{Total number of branches}} \times 100$$

- **Path testing:** Ensures that every possible path through the code is executed at least once in some test case.

$$\text{Path coverage (\%)} = \frac{\text{Number of execution paths executed}}{\text{Total number of execution paths}} \times 100$$

- **Basis Path testing:** It is a subset of path testing, where the goal is to identify and test a minimal set of independent paths that can cover all the possible decision points in the program.

$$\text{Basis Path coverage (\%)} = \frac{\text{Number of independent paths executed}}{\text{Total number of independent paths}} \times 100$$

Integration Testing

- Big-bang
- Bottom-up
- Top-down
- Sandwich testing

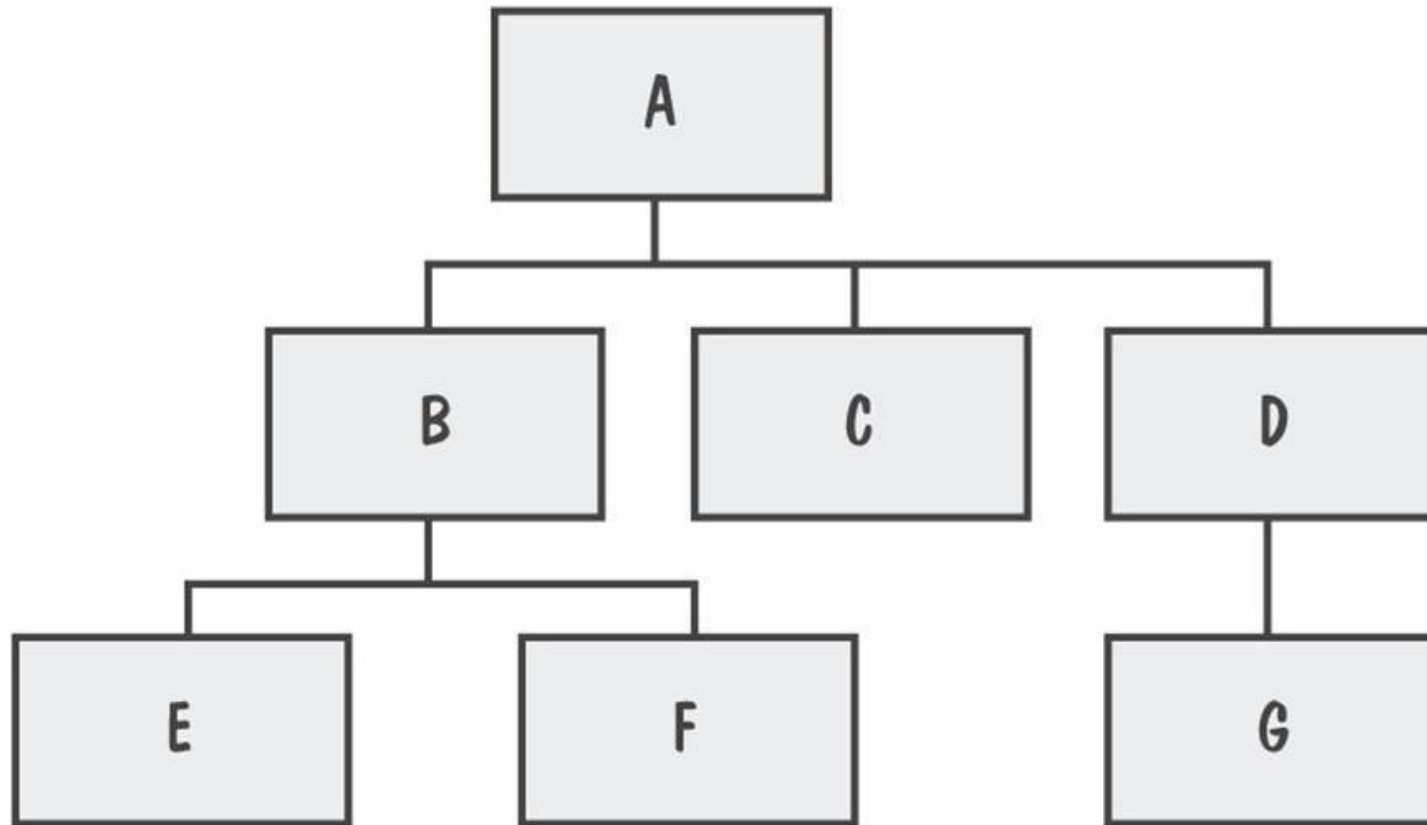
Integration Testing

Terminology

- Component Driver: a routine that calls a particular component and passes a test case to it
- Stub: a special-purpose program to simulate the activity of the missing component

Integration Testing

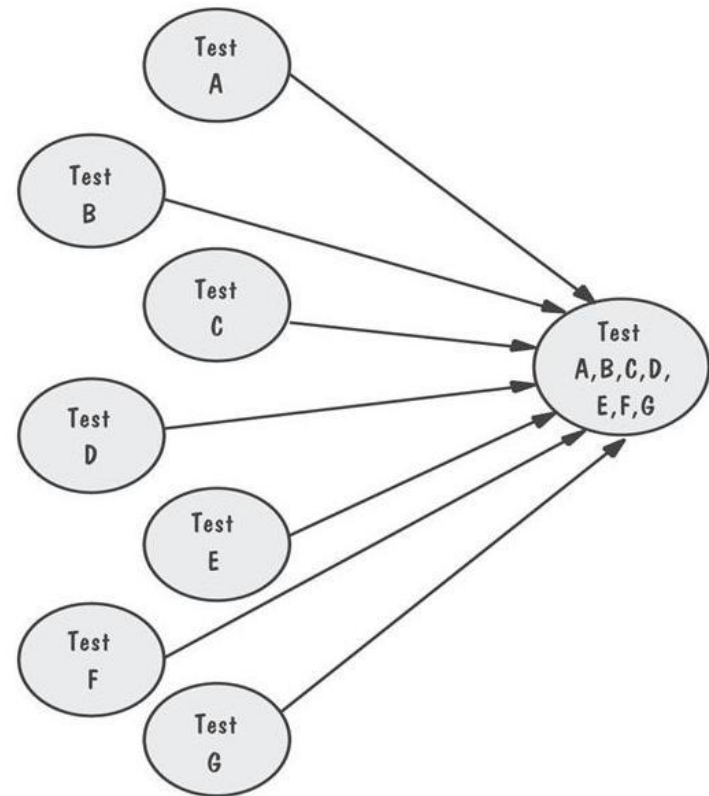
- System viewed as a hierarchy of components



Integration Testing

Bing-Bang Integration Example

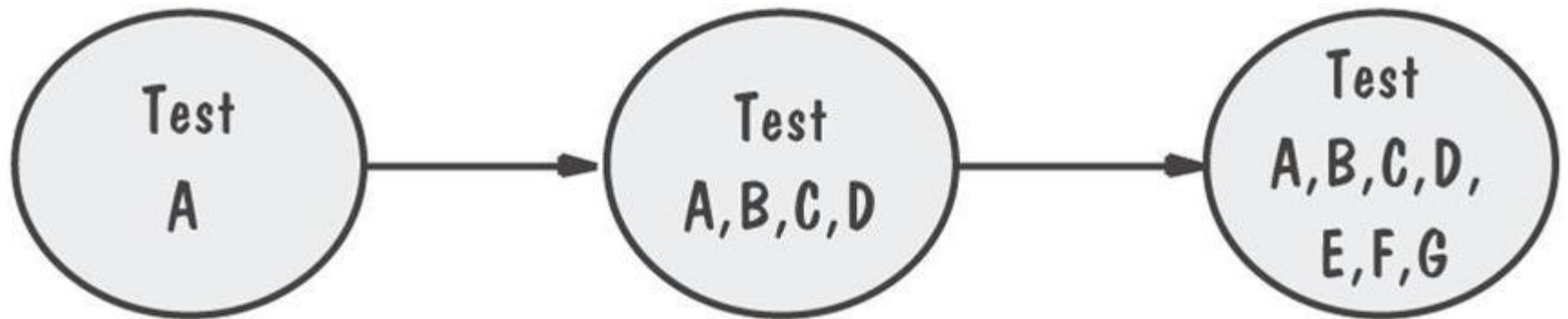
- All components integrated at



Integration Testing

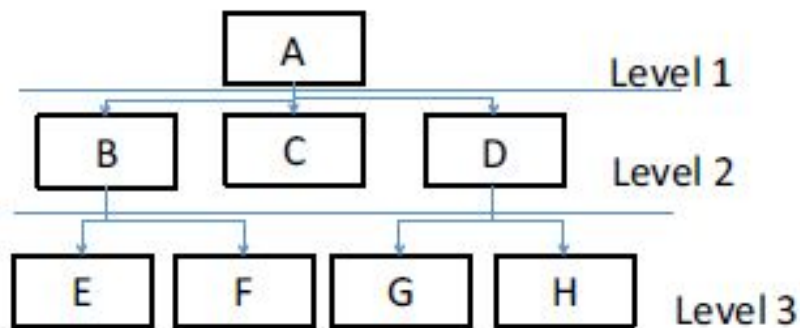
Top-Down Integration Example

- Only A is tested by itself
- Stubs of B, C and D are used at first level
- N-1 stubs required (N=Number of nodes)
- Locating faults?



Integration Testing

Top-Down Integration Example



Environments:

Session 1: A, stub(B), stub(C), stub(D)

S2: A, B, stub(C), stub(D)

S3: A, stub(B), C, stub(D)

S4: A, stub(B), stub(C), D

S5: A, B, stub(E), stub(F), C, D, stub(G), stub(H)

S6: A, B, E, stub(F), C, D, stub(G), stub(H)

S7: A, B, stub(E), F, C, D, stub(G), stub(H)

S8: A, B, stub(E), stub(F), C, D, G, stub(H)

S9: A, B, stub(E), stub(F), C, D, stub(G), H

S10: A, B, E, F, C, D, G, H

General formula:

Number of stubs: (nodes – 1)

Number of sessions: (non-leaves)+edges

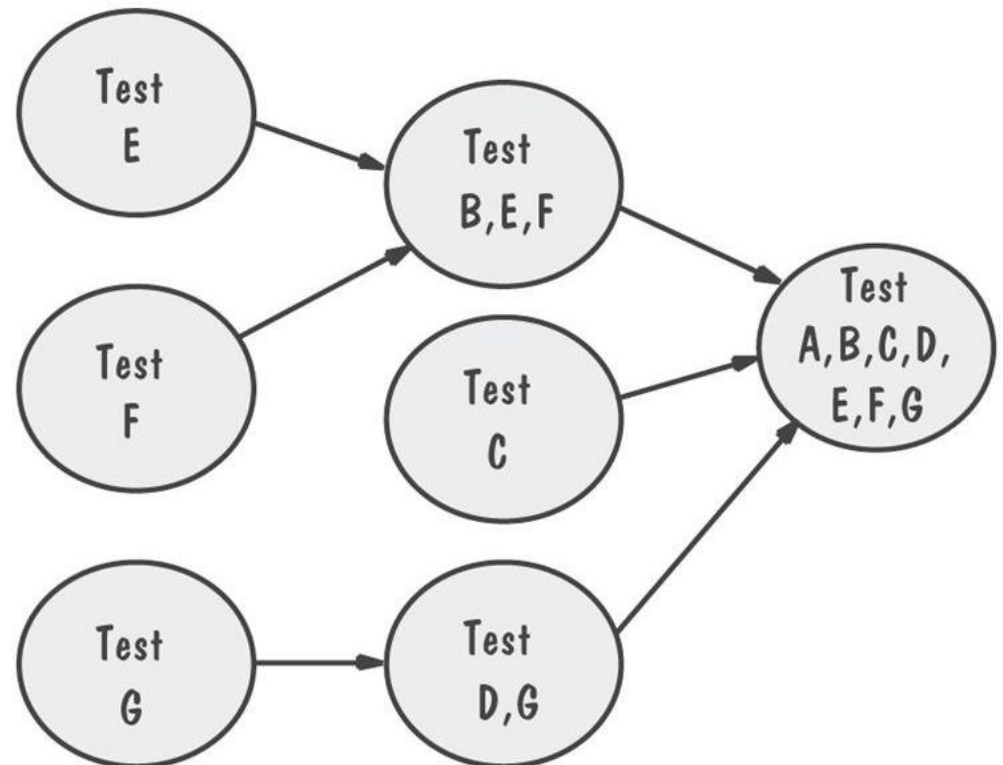
Number of stubs: ? 7

Number of sessions: ? 10

Integration Testing

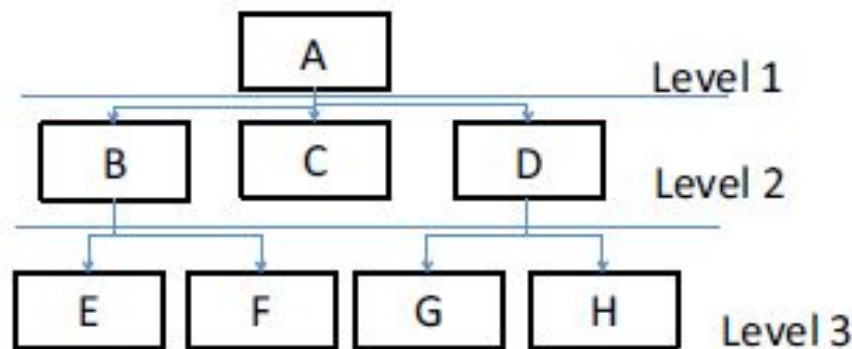
Bottom-Up Integration Example

- Drivers are used to call the child functions
- Drivers are relatively intelligent
- N - leaves drivers
- Locating faults?



Integration Testing

Bottom-Up Integration Example



Environments:

Session 1: E, driver(B)

S2: F, driver(B)

S3: E, F, driver(B)

S4: G, driver(D)

S5: H, driver(D)

S6: G, H, driver(D)

S7: E, F, B, driver(A)

S8: C, driver(A)

S9: G, H, D, driver(A)

S10: E, F, B, C, G, H, D, A

General formula:

Number of drivers: (non-leaves)

Number of sessions: (non-leaves)+edges

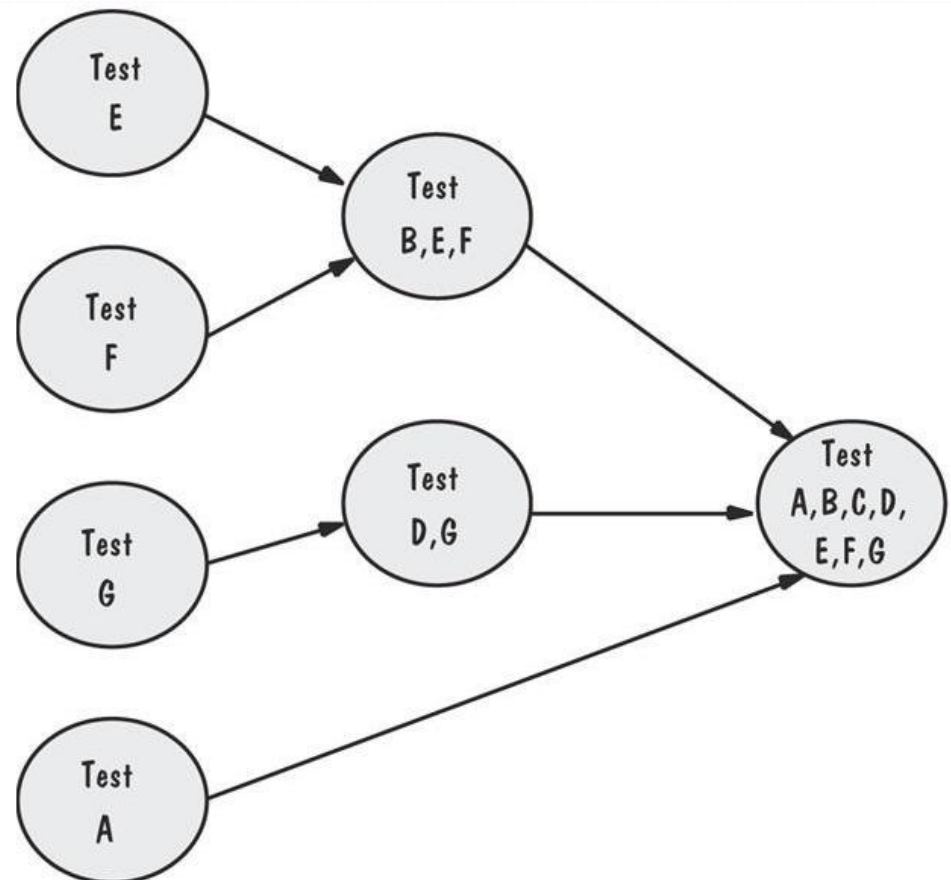
Number of drivers: ? 3

Number of sessions: ? 10

Integration Testing

Sandwich Integration Example

- Viewed system as three layers
- Employ BU where writing drivers is not costly
- Employ TD where stubs are easier to Write
- Locating faults?



System Testing

- System is tested as a whole
- Different types of testing considered during System Testing:
 - Functional Testing (GUI)
 - Scalability
 - Performance Testing
 - Sanity
 - Usability
 - Smoke
 - Load
 - Regression
 - Volume
 - Compatibility
 - Stress
 - Installation
 - Security
 - Ad hoc

Steps to do System Testing

The following steps are important to perform System Testing:

- Step 1: Create a System Test Plan
- Step 2: Create Test Cases
- Step 3: Carefully Build Data used as Input for System Testing
- Step 3: If applicable create scripts to
 - - Build environment and
 - - to automate Execution of test cases
- Step 4: Execute the test cases
- Step 5: Fix the bugs if any and re test the code
- Step 6: Repeat the test cycle as necessary

Principles of System Testing

Regression Testing

- Identifies new faults that may have been introduced as current one are being corrected
- Verifies a new version or release still performs the same functions in the same manner as an older version or release

Principles of System Testing

Regression Testing Steps

- Inserting the new code
- Testing functions known to be affected by the new code
- Testing essential function of m to verify that they still work properly
- Continuing function testing $m + 1$

Acceptance Tests

Purpose and Roles

- Enable the customers and users to determine if the built system meets their needs and expectations
- Written, conducted and evaluated by the customers

Acceptance Tests

Types of Acceptance Tests

- Pilot test: install on experimental basis
- Alpha test: in-house test
- Beta test: customer pilot
- Parallel testing: new system operates in parallel with old system

Result of Acceptance Tests

- List of requirements that
 - are not satisfied
 - must be deleted
 - must be revised
 - must be added

Installation Testing

- Before the testing
 - Configure the system
 - Attach proper number and kind of devices
 - Establish communication with other system
- The testing
 - to verify that the system has been installed properly and works

Software Testing Tools

- Automated Testing Tools
 - Selenium, QTP, SilkTest, WinRunner, LoadRunner, Jmeter
- Testing Management Tools
 - TestManager, TestDirector
- Bug Tracking/Configuration Management Tools
 - Bugzilla, Jitterbug, SilkRadar

When to Stop Testing

- No time left
- No money left
- Statistical Criteria
 - Number of defects found per week becomes lower than a set threshold

References

- UCF Slides
- Software Testing, A Craftsman's Approach by Jorgensen